Multiple Instance Learning: Literature Review, ABMIL Baseline, and Research Extensions

Syed Rayyan Albeez Ikramullah
M.Eng. Aerospace Engineering, University of Toronto
ECE1512 Project A Report
Code: https://github.com/Rayz8839/ece1512-projectA-mil

***Abstract*—Whole-slide images (WSIs) are gigapixel digital pathology images that usually carry a single label for the whole slide, which makes multiple instance learning (MIL) a natural framework for WSI classification. The literature review (Sections I–IX) covers MIL assumptions and representative methods, with attention-based MIL (ABMIL) as the baseline. Section X evaluates our implementation of the gated ABMIL head (the starter supplies the data loader, configurations and metric helpers) on CAMELYON16, CAMELYON17, and BRACS using frozen 384-dimensional patch features; a single run with seed 42 reached AUC 0.934, 0.839, and 0.804 and the report analyzes each component and the training behavior. Section XI proposes two extensions aimed at the overfitting and class imbalance seen in the baseline, implements one of them (a training-objective change combining class-weighted, label-smoothed cross-entropy, random patch dropout, and attention-entropy regularization), and compares it with the baseline on all three datasets.**

***Index Terms*—multiple instance learning, whole-slide images, computational pathology, attention mechanisms, weakly supervised learning**

I. INTRODUCTION

Pathologists traditionally examine thin tissue sections through a microscope. In modern workflows, these sections can be digitized as whole-slide images (WSIs), allowing them to be stored, shared, and analysed computationally. A single WSI may exceed $10^5 \times 10^5$ pixels [1]. Processing such an image directly with a conventional convolutional neural network (CNN) or transformer is therefore impractical. Reducing the image to a much smaller resolution is not a satisfactory solution either, because the fine cellular structures that often carry diagnostic information may disappear during downsampling.

Obtaining detailed annotations is another major obstacle. Marking tumour boundaries or other relevant regions on a large collection of slides requires considerable time from trained pathologists. In contrast, slide-level labels, such as whether a lymph node contains metastasis, are often available from existing clinical reports. The result is a weakly supervised learning problem: the image contains many potentially informative regions, but only the label for the complete slide is known.

Multiple instance learning (MIL) is well suited to this setting. The complete slide is treated as a bag, and the patches extracted from the slide are treated as its instances. During training, only the bag label is provided. The model must therefore learn which patches provide evidence for that

label and how those patches should be combined. Early MIL approaches generally relied on fixed aggregation rules, such as taking the largest instance score. More recent methods learn the aggregation function itself, often through attention mechanisms or models that capture relationships between patches.

The availability of strong pre-trained feature extractors has further changed the design of WSI models. In many current pipelines, the feature extractor is run once and the resulting patch embeddings are stored. Training then concentrates on the aggregator, which converts a variable-length collection of patch features into a single slide representation and prediction. This aggregator is the main component trained in the ABMIL baseline used in this project.

The remainder of this review is organized as follows. Section II formulates the MIL problem and introduces the standard assumption, alternative assumptions, and permutation invariance. Section III presents a taxonomy of MIL methods. Section IV reviews classical MIL approaches. Section V discusses deep MIL for histopathology, with a more detailed treatment of ABMIL and a brief review of later extensions. Section VI discusses feature representation, Section VII covers training and evaluation on the three project datasets, Section VIII discusses open problems and the link to this project, and Section IX summarises the literature. Section X reports the ABMIL baseline on the three datasets, and Section XI proposes two extensions and compares the implemented one with the baseline. Appendix A gives the reproducibility checklist.

II. PROBLEM FORMULATION

A. Bags, instances, and the standard assumption

In MIL, a bag is represented as a set

$$X = \{x_1, \ldots, x_N\}, \qquad x_i \in \mathcal{X},$$

where the number of instances, N, may vary from one bag to another. Each instance has an unobserved label $y_i \in \{0, 1\}$. The training data, however, provide only one label for the complete bag, denoted by Y.

The original MIL formulation introduced by Dietterich *et al.* [2] uses the standard assumption that a bag is positive whenever at least one of its instances is positive:

$$Y = \max_{i \in \{1,\ldots,N\}} y_i. \qquad (1)$$

This assumption is reasonable for some pathology tasks. For example, in lymph-node metastasis detection, the presence of even a small cluster of tumour cells can be sufficient to label the entire slide as positive.

B. Situations in which the standard assumption is insufficient

The maximum-instance rule does not describe every clinical classification problem. Alternative MIL assumptions have been discussed by Weidmann *et al.* [3] and Foulds and Frank [4]. Under a threshold-based formulation, a bag becomes positive only when at least θ positive instances are present. Under a count-based formulation, the number of positive instances must fall within a specified range. In a collective formulation, the label depends on the overall combination of instance types rather than on a single patch.

These alternatives are relevant to the datasets considered in this project. In CAMELYON17, the slide categories—negative, isolated tumour cells, micro-metastasis, and macro-metastasis—are related to the size of the largest tumour deposit. Consequently, identifying one suspicious patch is not sufficient; the model must also capture information about the extent of the tumour. BRACS contains seven lesion categories, several of which, including atypical hyperplasia and ductal carcinoma in situ, may be difficult to distinguish and may occur alongside benign tissue [5]. A single maximum score cannot fully represent these cases. Learned aggregation is useful because it can combine evidence from multiple regions instead of relying on one selected patch.

C. Permutation invariance

The patches in a WSI bag do not have a natural sequence. Reordering the patches should therefore leave the slide prediction unchanged. If $\pi(X)$ denotes a permutation of the instances, a valid bag-level function S should satisfy

$$S(\pi(X)) = S(X).$$

Zaheer *et al.* [6] showed that a permutation-invariant function can be expressed in the form

$$S(X) = \rho\left(\sum_{i=1}^{N} \phi(x_i)\right) \qquad (2)$$

for suitable functions ϕ and ρ. Ilse *et al.* [7] applied this idea to neural MIL. In practical terms, a neural MIL model can be separated into three components: an embedding function that maps each patch to a feature vector, an order-independent pooling operation, and a classifier that produces the bag prediction. The methods discussed in this review mainly differ in how they implement these components and how much information they retain during aggregation.

III. TAXONOMY OF MIL METHODS

Amores [8] classified MIL approaches according to the space in which the bag-level decision is made. Later surveys adopted a similar organization [9]–[11], and the same taxonomy is used here. Fig. 1 places these approaches within a typical weakly supervised pathology pipeline.

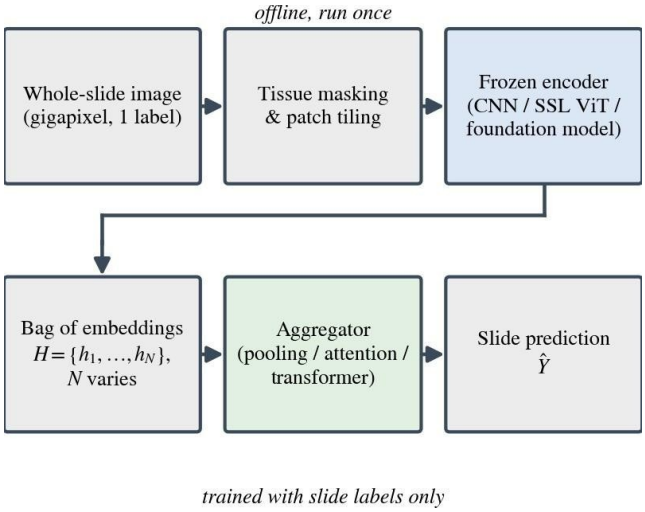


Fig. 1. Standard weakly supervised pipeline for WSI classification. A feature encoder is commonly frozen and applied once to all patches, so the subsequent training stage updates only the aggregation module and the bag-level classifier. In this project, the stored feature files represent the input to the bag-level stage.

Instance-space (IS) methods. An instance classifier assigns a score to every patch, after which a fixed rule—such as maximum, mean, or top-k pooling—combines the scores. These methods require only bag labels, but the instance classifier receives indirect and often noisy supervision. Their main advantage is interpretability: the individual scores can be used to identify regions that contributed to the prediction.

Embedding-space (ES) methods. Each instance is first mapped to an embedding. The embeddings are then pooled into one bag representation, which is passed to a classifier. Unlike instance-space methods, these models do not need to classify each patch separately.

Bag-space (BS) methods. These methods compare complete bags directly, using a distance or a kernel defined over sets. A conventional classifier, such as an SVM or k-nearest-neighbour classifier, can then operate on the resulting comparisons.

ABMIL, TransMIL, and most recent WSI MIL models are embedding-space approaches with learned pooling functions.

CLAM also operates primarily in embedding space, although it adds an auxiliary instance-level objective.

IV. CLASSICAL MIL

Dietterich *et al.* [2] formalized the problem of learning from labeled bags of instances, which gave rise to MIL. In that setting, a molecule was represented as a bag of possible shapes, or instances, and the molecule was considered active if at least one shape could bind to the target. Their method searched for an axis-aligned rectangle in feature space that included at least one instance from every positive bag while excluding instances from negative bags.

Diverse Density [12], proposed by Maron and Lozano-Pérez, followed as an early framework for locating the instance concept that best explains the bag labels. Rather than searching for a rectangle, this method looks for a point in feature space that is close to instances from many positive bags and far from instances in negative bags. Wang and Zucker [13] introduced Citation-kNN, a nearest-neighbour method designed to compare sets of instances. Gärtner *et al.* [14] developed kernels for bags, making it possible to apply support vector machines directly to MIL problems.

Andrews *et al.* [15] proposed mi-SVM and MI-SVM. These methods alternate between estimating instance labels that satisfy the standard MIL assumption and training an SVM using those labels. MI-SVM places particular emphasis on the most extreme instance in each bag.

Classical methods provide important foundations, but they are not an ideal fit for modern WSI classification. They generally depend on manually designed features, and their computational cost can become substantial when a slide contains thousands of patches. Nevertheless, their central ideas—latent instance labels, set-level comparisons, and aggregation under weak supervision—remain visible in later neural approaches.

V. DEEP MIL FOR HISTOPATHOLOGY

A. Pooling and early deep models

Neural MIL makes it possible to learn instance representations and the bag-level aggregation function jointly. Wang *et al.* [16] compared two early neural formulations: mi-Net, which predicts at the instance level before pooling the scores, and MI-Net, which pools the learned instance embeddings before classification. In their experiments, the embedding-based formulation generally performed better.

Common pooling strategies include maximum pooling, mean pooling, and smooth alternatives such as log-sum-exp pooling [17]. Both maximum and mean pooling have important weaknesses for WSI analysis. With maximum pooling, the gradient is passed through only the selected patch, which can make optimization unstable and can reinforce an incorrect early choice. Mean pooling has the opposite problem: because normal tissue often occupies most of a slide, the representation of a small tumour region may be overwhelmed by the much larger amount of irrelevant tissue. In addition, neither rule learns how to weight the instances according to the task.

Several early pathology systems attempted to learn useful patch-level evidence directly from slide labels. Hou *et al.* [18] trained a patch-level CNN using an expectation–maximization-style procedure. Their method alternated between selecting discriminative patches and updating the classifier. Campanella *et al.* [19] worked with a very large slide cohort. They first trained a CNN by assigning the slide label to the highest-ranked tile and then used a recurrent neural network (RNN) to process the most suspicious tiles. Their results showed that slide-level supervision can produce strong area-under-the-curve performance when very large training datasets are available.

The approach, however, requires substantial data and computational resources. For this reason, later work has often adopted frozen feature extractors followed by relatively lightweight aggregation modules.

B. Attention-based MIL (ABMIL)

Ilse *et al.* [7] proposed learning the pooling operation through attention. Let

$$h_i = g(x_i) \in \mathbb{R}^L$$

be the embedding of instance *i*. ABMIL forms a bag representation as a weighted sum of the instance embeddings:

$$z = \sum_{i=1}^{N} \alpha_i h_i, \qquad \alpha_i = \frac{\exp(a_i)}{\sum_{j=1}^{N} \exp(a_j)}. \tag{3}$$

In the basic attention formulation, the score for an instance is

$$a_i = w^\top \tanh(V h_i).$$

The gated version uses two branches and defines the score as

$$a_i = w^\top \left[\tanh(V h_i) \odot \sigma(U h_i)\right], \tag{4}$$

where the learned parameters have dimensions

$$V, U \in \mathbb{R}^{D_a \times L}, \qquad w \in \mathbb{R}^{D_a}$$

and σ denotes the sigmoid function (bias terms are omitted from Eq. (4) for clarity but are included in the parameter counts of Table III). The elementwise product is written as $\odot$. The additional gate introduces a second nonlinear pathway. This is useful because the hyperbolic tangent can behave almost linearly over part of its input range, limiting what a single branch can represent. Related gating ideas have

also been used in sequence models [20]. Although the use of softmax resembles the attention mechanism introduced by Bahdanau *et al.* [21], ABMIL applies the mechanism to an unordered set rather than to a sequence.

The weighted representation z is passed to a bag-level classifier, and the model is trained using the slide label, typically with cross-entropy loss. The attention operator has several useful properties. Because the weights are computed from a set of instances and are summed in an order-independent way, the output does not depend on patch ordering. The mechanism also accepts bags with different numbers of instances and is differentiable, allowing the embedding, attention, and classifier parameters to be optimized together. The weights are non-negative and sum to one. Equal attention weights reproduce mean pooling, whereas a single dominant score makes the behaviour approach maximum pooling. Finally, the attention values can be projected back onto the slide to produce a heat map indicating which regions received the most weight. Such maps are useful for interpretation, although they should not automatically be treated as definitive lesion boundaries.

1) Why attention can be easier to optimize than maximum pooling: Let $g = \partial L/\partial z$ be the gradient of the loss with respect to the bag representation. Differentiating Eq. (3) through the softmax gives

$$\frac{\partial z}{\partial a_i} = \alpha_i (h_i - z), \qquad \frac{\partial \mathcal{L}}{\partial a_i} = \alpha_i \, g^\top (h_i - z). \tag{5}$$

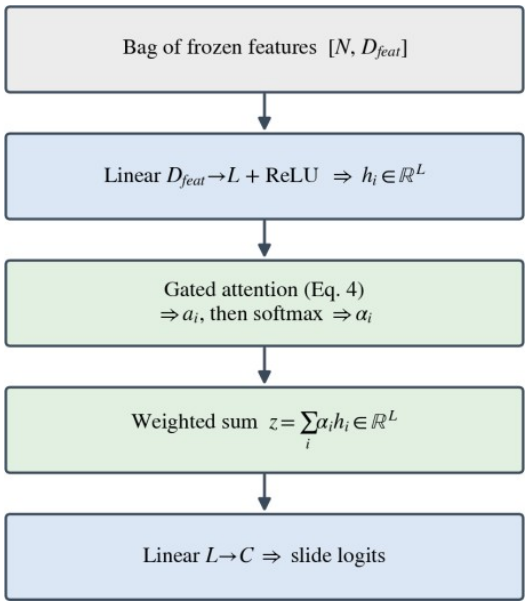


Fig. 2. Gated ABMIL architecture. The default configuration maps the 384-dimensional input features to an embedding dimension of L = 256, uses an attention dimension of Da = 128, and has one attention branch. Only three small modules (the feature projection, the gated attention module, and the bag-level classifier) are learned after the patch features have been extracted.

This expression highlights two differences between attention and maximum pooling. First, every patch with a non-zero attention weight can receive a gradient, whereas maximum pooling updates only the selected patch. Second, the attention score increases when

$$g^\top (h_i - z) < 0,$$

meaning that moving the bag representation toward h_i would reduce the loss. In this way, the model can increase the weight of patches that differ from the current average content in a direction that is useful for classification. This behaviour is particularly relevant when a small tumour region is surrounded by a large amount of normal tissue.

Attention also has a weakness. The gradient is scaled by α_i, so an instance that receives a very small initial weight may learn very slowly. Over time, the model can therefore become concentrated on a small number of patches. This attention-collapse or attention-concentration problem motivates several of the later methods discussed below.

C. Extensions of attention-based aggregation

Table I summarizes several representative methods. Each method modifies attention-based aggregation to address a particular limitation, including weak localization, dependence on one critical patch, noisy pseudo-bags, or excessive concentration of attention.

CLAM. Lu *et al.* [22] extended gated attention to multi-class classification by assigning a separate attention branch and classifier to each class. This allows the model to focus on different slide regions depending on the class under consideration. CLAM also introduces an instance-level clustering objective. Patches with the highest and lowest attention values are treated as pseudo-positive and pseudo-negative examples, respectively, and a small instance classifier is trained with a smooth-SVM loss. The method can be effective on relatively small training sets and provides attention maps that support coarse localization. Its instance labels are nevertheless derived from attention, so errors in the initial attention ranking can produce incorrect pseudo-labels.

DSMIL. Li *et al.* [23] proposed a two-stream architecture. One stream identifies a critical instance using maximum pooling. The second stream computes attention for all instances according to their similarity to that critical instance. The method also makes use of contrastive self-supervised features and features extracted at multiple magnifications. Its reliance on a dominant critical instance can become a limitation when the diagnostic evidence is distributed across several regions rather than concentrated in one patch.

DTFD-MIL. Zhang *et al.* [24] addressed the limited number of slide-level training examples by dividing each slide into several randomly sampled pseudo-bags. Every pseudo-bag inherits the label of the original slide, which increases the number of training units. A first-stage attention model is trained on these pseudo-bags, followed by a second-

stage model that operates on distilled features. Randomly generated pseudo-bags may contain little or no tumour despite receiving a positive slide label, creating label noise. The second stage is intended to reduce the effect of this noise through feature distillation.

ACMIL. Zhang *et al.* [25] argued that attention models can overfit by repeatedly selecting a small group of easy patches, particularly when the training set is small. ACMIL uses multiple attention branches and encourages the branches to learn different patterns. It also randomly masks highly attended patches, forcing the model to consider additional evidence. This strategy directly addresses the slow-learning and concentration behaviour suggested by Eq. (5), although it introduces additional hyperparameters and design choices.

D. Methods that model relationships between patches

ABMIL assigns an attention score to each patch independently. As a result, the basic model does not explicitly use patch coordinates or the relationship between neighbouring regions. TransMIL [26] introduces self-attention between patches and uses the Nyström approximation [27] to reduce the computational cost of this operation. It also incorporates a positional encoding module based on conditional position encoding [28] so that the model can use information about the two-dimensional slide layout.

Other methods explore different ways to model contextual information. RRT-MIL [29] re-embeds patch features with a transformer-like module before applying attention pooling. MambaMIL [30] uses state-space layers with the goal of achieving linear computational complexity. HIPT [31] builds a hierarchy of vision transformers that processes information

at the cell, region, and slide levels. Graph-based approaches, including the method of Zhao *et al.* [32], connect spatially nearby patches and use graph convolution to exchange information between them.

TABLE I
SUMMARY OF REPRESENTATIVE MIL METHODS FOR SLIDE CLASSIFICATION

Method	Space	Core idea	Main limitation
Diverse Density [12] (1998)	IS	Searches for a concept point close to instances from many positive bags and far from negative bags	Non-convex optimization, hand-crafted features, limited scalability
mi-SVM / MI-SVM [15] (2003)	IS / ES	Margin-based learning with latent instance labels constrained by the standard MIL assumption	Iterative optimization, quadratic kernel cost, fixed input features
mi-Net / MI-Net [16] (2018)	IS / ES	Learns instance scores or instance embeddings before maximum or mean pooling	Pooling is fixed and cannot adapt to the data
ABMIL [7] (2018)	ES	Softmax attention over instance embeddings, with optional tanh ⊙ σ gating	Instances scored independently; attention may concentrate
Campanella et al. [19] (2019)	IS + ES	Top-ranked tiles train a CNN; selected tiles are then processed by an RNN	Needs very large cohorts and heavy computation
CLAM [22] (2021)	ES + IS	Class-specific attention branches and instance clustering with pseudo-labels	Attention-derived pseudo-labels may be inaccurate
DSMIL [23] (2021)	ES	Critical-instance stream, similarity-based attention, multi-scale SSL features	Assumes one dominant critical instance is informative
TransMIL [26] (2021)	ES	Self-attention across patches with positional encoding	More demanding; prone to overfitting on small datasets
DTFD-MIL [24] (2022)	ES	Pseudo-bags with two-stage feature distillation	Pseudo-bags inherit potentially incorrect labels
ACMIL [25] (2024)	ES	Multiple attention branches and stochastic top-k masking against attention collapse	Extra hyperparameters and training decisions

"Space" follows the taxonomy of Section III (IS: instance space; ES: embedding space).

Relational models may capture structures that are difficult to represent with independent patch scoring, such as the boundary between tumour and stroma. However, they generally require more parameters and memory than basic ABMIL. Their additional flexibility can also increase the risk of overfitting, especially when only a small number of slides is available for training.

VI. FEATURE REPRESENTATION

The aggregation module cannot recover information that is absent from the patch features, which makes the choice of encoder an important part of the WSI pipeline. Early studies commonly relied on CNNs pre-trained on ImageNet, including ResNet-based encoders [33]. However, natural-image datasets and H&E-stained tissue images have very different visual characteristics. As a result, features learned from natural images may not capture all of the cellular and morphological patterns that are relevant to pathology.

Self-supervised learning provides a way to pre-train an encoder on pathology images without requiring detailed manual labels for every patch. Contrastive approaches based on SimCLR [34] have been adapted to histology images [23], [35]. Kang *et al.* [36] also examined self-distillation with vision transformers, including DINO [37] and ViT [38], across a range of pathology datasets. These methods aim to learn representations from the structure and variation within the images themselves before the slide-level classification task is introduced.

The feature files supplied for this project contain 384-dimensional vectors. This dimensionality is consistent with the

output of a ViT-S/16 configuration, as noted in the project configuration files. In the experiments here, the feature extractor is kept fixed, so the comparison focuses on how effectively different MIL aggregators use the same input representation.

More recently, pathology foundation models have been trained using very large collections of tissue images and slides. Examples include UNI [39], CONCH [40], Virchow [41], and Prov-GigaPath [42]. Stronger encoders may reduce the performance gap between aggregation methods because the input features already contain more useful information. Consequently, a new aggregator should be compared against a carefully tuned simple baseline rather than against an under-optimized baseline. To make the comparisons in this project fair, the features, data splits, and model-selection rule remain unchanged across experiments, as required by the project specification.

VII. TRAINING AND EVALUATION

A. Datasets

The project uses three public datasets, summarized in Table II. CAMELYON16 [43] contains lymph-node WSIs that are labelled according to whether breast-cancer metastasis is present. It provides an official training/test division of 270 and 129 slides, respectively, together with tumour annotations that can be used to assess localization. CAMELYON17 [44] extends this setting to slides collected from five medical centres. Its labels distinguish negative slides, isolated tumour cells, micro-metastases, and macro-metastases according to the size of the metastatic deposit. Because the participating centres use different scanners and staining procedures, the dataset also provides a useful test of domain shift.

BRACS [5] is a breast-lesion dataset with seven diagnostic categories. These categories can also be grouped into broader benign, atypical, and malignant classes. Taken together, the three datasets represent different types of classification problem: binary detection in CAMELYON16, metastasis staging in CAMELYON17, and fine-grained multi-class diagnosis in BRACS.

B. Loss and optimization

Most MIL classifiers are trained using slide-level cross-entropy. For a slide X with label Y, the loss can be written as

$$\mathcal{L} = -\log p_\theta(Y \mid X),$$

where $p_\theta(Y \mid X)$ is the probability assigned to the observed label by the model. Adam-type optimizers are commonly used for this purpose. AdamW [45] separates weight decay from the adaptive gradient update, making the effect of this regularization easier to control.

A WSI may contain a different number of patches from every other slide. One practical solution is to process one slide at a time. Another is to pad bags to a common length and use a mask. When padding is used, the attention softmax must also be masked. Otherwise, the padded zero vectors can receive attention weight and influence the bag representation even though they do not correspond to real tissue patches.

The project YAML configurations include a learning-rate warm-up key, but our training loop does not use it, so the learning rate stays constant in our experiments. A warm-up would make the early stages of training more stable, when the attention distribution is close to uniform and the model has not yet learned which patches are useful. Dropout, weight decay, and selecting the best checkpoint according to validation performance are standard ways to reduce overfitting. The issue becomes more serious in few-shot experiments, where only k slides per class are used and the result can depend strongly on the random seed.

C. Metrics

For binary classification, AUROC is commonly reported because it evaluates ranking performance across all possible thresholds. It is also generally less sensitive to class imbalance than raw accuracy. For multi-class classification, macro-averaged AUROC and macro F1 give every class the same importance. This prevents a large class from hiding poor performance on smaller classes. Accuracy remains useful as an additional measure, but it should be interpreted alongside the class distribution and per-class results.

CAMELYON16 also supports the free-response receiver operating characteristic (FROC) metric for lesion detection. Unlike a slide-level metric, FROC requires patch- or lesion-level prediction scores. Calibration is another relevant consideration for clinical deployment because a model should produce probabilities that correspond reasonably to the observed frequency of outcomes. In practice, however, calibration is reported less consistently than discrimination metrics such as AUROC and F1.

D. Common problems in evaluation

Several evaluation choices can produce misleadingly strong results if they are not handled carefully.

Test-set leakage. Selecting epochs, thresholds, or hyperparameters using test performance leads to optimistic estimates. The best checkpoint should be selected using the validation set, and the test set should be evaluated only after the model and training configuration have been finalized.

Patient leakage. CAMELYON17 contains multiple slides from the same patient. A random split at the slide level may therefore place highly similar tissue from one patient in both the training and test sets. Splitting by patient or by centre reduces this risk. The supplied data loader keeps the test centres (3 and 4) separate from training, but it draws the validation slides at random from the remaining slides, so the C17 validation set is not patient-disjoint from the training set (Section X-A).

Small test sets. BRACS contains only 86 test slides across seven classes. A single incorrectly classified slide can change the accuracy by more than one percentage point, and some categories contain very few examples. Results should therefore be reported across several random seeds, with bootstrap confidence intervals where possible, before claiming that one method is superior to another. Our own experiments use a single seed and do not meet this standard (Sections X-E and XI-F).

Unequal hyperparameter tuning. A comparison is not fair if the baseline is run with default settings while the proposed method receives extensive tuning. Both approaches should be given comparable tuning budgets, training conditions, and opportunities to select hyperparameters.

E. Evaluation of attention maps

Attention heat maps are widely used to illustrate the regions that influence a MIL prediction. However, whether attention weights constitute a faithful explanation remains debated. Jain and Wallace [46] reported that attention weights did not always correspond to feature importance in natural-language processing models, while Wiegreffe and Pinter [47] argued that the conclusion depends on the definition of explanation being used.

For pathology applications, numerical tests provide stronger evidence than visual inspection alone. One possible evaluation is to compare the regions receiving high attention with the expert tumour annotations available for CAMELYON16. Another is an ablation experiment: remove the patches with the highest attention and measure the resulting change in the slide prediction, then compare it with the effect of removing randomly selected patches. If the model depends on the highlighted regions, removing them should generally have a larger effect.

Attention values should also be interpreted within the context of a single slide. Because the weights are normalized relative to the other patches in that bag, a high percentile indicates that a patch received more attention than most patches on the same slide. It does not mean that the patch has

a high probability of containing tumour. The starter notebook makes the same distinction.

TABLE II
DATASETS AND SPLITS USED BY THE STARTER PACKAGE

Dataset	Classes	Slides	Split
C16	2 (normal / tumour)	398 (supplied)	Official test slides held out; 10% of the remaining slides used for validation
C17	4 (negative, ITC, micro-, macro-metastasis)	499	Centre-based: centres ≥ 3 for testing; 269 train / 30 val / 200 test
BRACS	7 (or 3 / 2 after grouping)	537	Predefined: 389 train / 62 val / 86 test

C17 and BRACS counts are from the supplied metadata. For C16, the test set is identified by slide IDs containing "test"; the supplied file holds 269 non-test slides (242 training, 27 validation), one fewer than the 270 official training slides.

VIII. OPEN PROBLEMS AND CONNECTION TO THIS PROJECT

Attention concentration. The softmax function assigns a positive weight to every non-masked instance, although most patches in a WSI may contain little information for the task. Sparsemax [48] offers an alternative that can assign exactly zero weight to some instances. Sparsity is not automatically beneficial, however. ACMIL suggests that excessive concentration on a small number of easy patches can also reduce generalization. The most useful level of sparsity is therefore likely to depend on the dataset, the label structure, and how widely the diagnostic evidence is distributed across the slide.

Context versus instance independence. Basic ABMIL scores each patch largely independently. Models that allow patches to interact may perform better when the diagnosis depends on tissue architecture or spatial context. For example, the relationship between tumour, stroma, and surrounding tissue may be informative even when none of the individual patches is decisive by itself. On the other hand, the datasets used in this project contain only a few hundred slides, so the additional parameters in contextual models may increase the risk of overfitting. It remains an empirical question whether explicit context provides a consistent benefit when the patch features are already strong.

Domain shift. Differences in staining protocols, scanners, institutions, and patient populations can reduce model performance when a trained system is applied to a new domain. The centre-based organization of CAMELYON17 provides a direct test of this issue. A model that performs well on slides from one centre may still struggle when the image appearance changes across centres.

Faithfulness of model explanations. Training from slide-level labels alone does not guarantee that an attention map will identify the actual tumour region. The model may instead rely on staining patterns, tissue preparation artefacts, or other correlations that distinguish the training data. Attention maps should therefore be treated as diagnostic evidence about model behaviour, not as proof that the highlighted regions are clinically correct. Whenever expert annotations are available, localization and deletion-based tests should supplement qualitative heat-map inspection.

Baseline model. The project baseline is the gated ABMIL model shown in Fig. 2. A linear layer followed by a ReLU maps the original 384-dimensional patch features to 256 dimensions. The gated attention module contains 128 hidden units and produces one attention score for each patch. These scores are normalized with a softmax, and the resulting weighted sum of patch features is passed to a final linear classifier.

The model is trained with cross-entropy loss and AdamW. The checkpoint with the best validation score is retained, while the test set is used once at the end of the experiment. Under the taxonomy introduced earlier, the baseline is an embedding-space MIL model with a single attention branch. This makes it a useful reference point for extensions involving multiple attention branches, sparse attention, instance-level auxiliary losses, contextual modelling, or alternative training objectives.

IX. SUMMARY OF THE LITERATURE

Multiple instance learning began as a way to solve weakly supervised problems such as drug-activity prediction and has since become a standard framework for learning from slide-level labels in computational pathology. Attention-based pooling is especially useful because it is differentiable, independent of patch ordering, compatible with variable-sized bags, and capable of producing a coarse indication of the regions used by the model.

Subsequent methods have addressed several weaknesses of basic ABMIL. Class-specific attention branches allow different diagnostic categories to use different regions. Pseudo-bag and feature-distillation strategies attempt to make better use of limited slide-level data. Multi-branch regularization methods reduce the tendency to focus repeatedly on a few easy patches, while transformer and graph-based models introduce relationships between spatially related instances.

The literature also shows that model architecture is only one part of a reliable comparison. The feature extractor, data split, validation procedure, random seed, and evaluation metric can all affect the result. For the experiments in this project, the same input features and data splits are used across models, checkpoint selection is based only on validation data, and the test sets are kept separate from training (official split for C16, centre-disjoint for C17, supplied split for BRACS). Two limitations remain. The C17 validation slides are drawn at random at the slide level, so they share patients with the training slides (Sections VII-D and X-A), which makes C17

checkpoint selection optimistic. The experiments also use a single seed (42), with multiple seeds left as future work, so their results should be read as point estimates (Sections X-E and XI-F). These practices provide the basis for comparing the ABMIL baseline with the extension.

TABLE III
PARAMETERS PER COMPONENT (2-CLASS MODEL)

Layers	Parameters
Linear(384,256) + ReLU	98,560
Gated attention (V, U, w), 128-d	65,921
Linear(256, #classes)	514
	164,995

X. BASELINE EVALUATION SUMMARY

A. Model Structure and Training Procedure

Input. Each slide is a bag of patch embeddings of dimension 384 taken from the provided HDF5 files; slides contain on average about 12.5k (C16), 10.6k (C17) and 9.0k (BRACS) patches. Only slide-level labels are available and the feature extractor is frozen, so only the MIL head is trained. The head has three parts (Table III): an encoder (Linear $384 \to 256$ + ReLU, applied per patch); a gated-attention aggregator [7], $a_i = \mathrm{w}^\top[\tanh(V h_i) \odot \mathrm{sigm}(Uh_i)]$ with 128 hidden units and a softmax over the unmasked patches of a slide, which produces one weighted slide embedding; and a one-layer linear classifier ($256 \to$ number of classes). The head has 164,995 trainable parameters in the 2-class case (165,509 for C17 and 166,280 for BRACS). *Training.* Cross-entropy loss, AdamW (learning rate 1e-4, weight decay 1e-5), batch size of one slide, 30 epochs, seed 42. After each epoch the model is evaluated on a validation split and scored by AUC + macro-F1; the checkpoint with the highest score (first one in case of ties) is restored and evaluated once on the test split. CAMELYON16 keeps its official test slides, CAMELYON17 uses centers ≥3 as test, and 10% of the remaining slides form the validation set; BRACS uses the supplied split. In C16 and C17 the validation slides are drawn at random (stratified, seed 42) from the non-test slides. For

C17 this is a slide-level split: replicating the loader with the supplied case_id column shows that all 30 validation slides come from patients (27) that also contribute slides to training, so C17 validation scores are optimistic and checkpoint selection can be affected. We kept the supplied split so that the comparison stays like for like. Test sets are small and strongly imbalanced (Table IV).

Metrics. We report accuracy, macro-averaged F1, cross-entropy loss and AUC. AUC is computed from the softmax probabilities by the supplied classification_metrics helper; for the multi-class tasks (C17, BRACS) it is the macro-averaged one-versus-all AUC, a common convention for multi-class AUROC.

TABLE IV
SLIDE COUNTS AND CLASS DISTRIBUTION (LABELS 0..K−1)

Split	Slides
Train	242
Val	27
Test	129
Train	269
Val	30
Test	200
Train	389
Val	62
Test	86

TABLE V
TEST PERFORMANCE OF ABMIL (SINGLE RUN, SEED 42)

Best val ep.	Acc (%)	AUC	Macro-F1
3	88.37	0.9342	0.8668
22	78.50	0.8391	0.5323
6	39.53	0.8042	0.3494

B. Baseline Performance

Table V summarises the test results of the selected checkpoints. Accuracy carries wide uncertainty on these small test sets: Wilson 95% intervals are 81.7–92.8% (C16, 114/129), 72.3–83.6% (C17, 157/200) and 29.9–50.1% (BRACS, 34/86), and one slide moves accuracy by 0.78, 0.5 and 1.16 points respectively. AUC and macro-F1 are given without intervals because per-slide test probabilities were not saved for a bootstrap. Fig. 3 shows loss, accuracy and validation AUC/F1 per epoch and Fig. 4 the test confusion matrices. The training loss and accuracy in Fig. 3 are running averages over each epoch, accumulated while the weights are still being updated, rather than a separate pass of the finished model over the training slides, so they tend to understate the true train/validation gap.

CAMELYON16. Training loss falls below 0.04 and validation loss below 0.004 by epoch 10, approaching zero thereafter, and validation AUC and F1 reach 1.0 from epoch 3 on. Because the validation set has only 27 slides, every epoch from 3 on reaches the maximum score (AUC 1.0 plus F1 1.0), so selection reduces to the first-epoch tie rule and carries no information about which later epoch would generalise best. On the test set the model makes no false positives (precision 1.00 for class 1) but misses 15 of 49 positive slides (recall 0.69), which explains the 88.4% accuracy and the gap to the perfect validation score. Besides the small validation set, the official test slides may also differ in distribution from the training slides; we did not test this.

CAMELYON17. Training accuracy keeps rising to about 99%, whereas validation loss is lowest at epoch 9 and then becomes noisy and drifts upward; validation accuracy fluctuates between roughly 70% and 90% after the first epochs. The majority class and the largest abnormal class are handled

well (F1 0.92 and 0.78), but classes 1 and 2 are weak (F1 0.24 and 0.19); 12 of the 18 class-2 test slides are predicted as class 3. The gap between accuracy (78.5%) and macro-F1 (0.53) reflects this imbalance. Note that the validation set contains only 2, 4 and 4 slides of classes 1–3, so checkpoint selection (epoch 22) is very noisy. Because validation slides share patients with the training set (Section X-A), epoch 22 should be read as a noisy choice, not a tuned one.

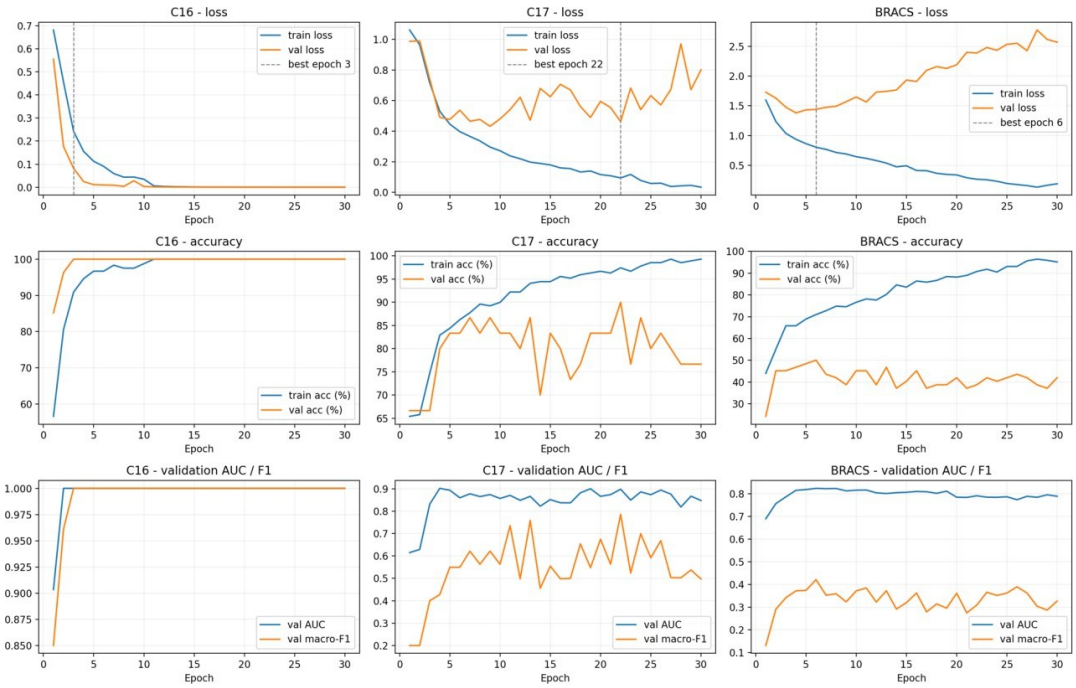


Fig. 3. Training/validation curves per dataset (columns: C16, C17, BRACS; rows: loss, accuracy, validation AUC/macro-F1). Dashed line: selected epoch (top row).

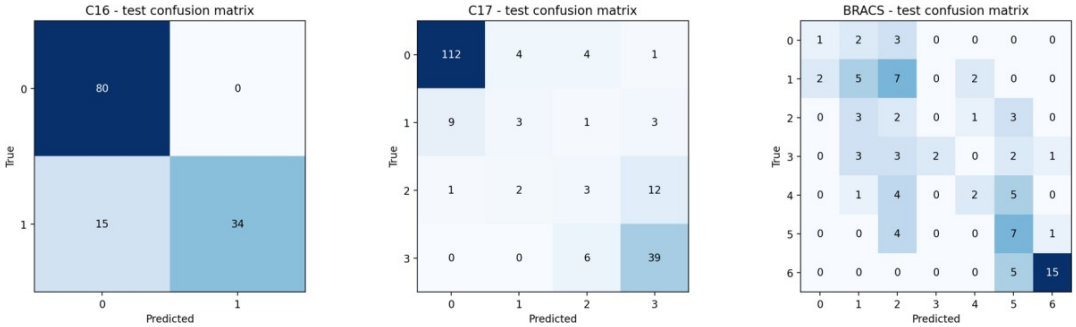


Fig. 4. Test confusion matrices (rows: true class, columns: predicted class).

BRACS. This is the hardest task: validation loss is lowest at epoch 4 and then rises from about 1.4 to about 2.6 while training accuracy reaches about 95% and validation accuracy stays near 40%, a clear case of overfitting with 389 training slides and 7 classes. Only class 6 (F1 0.81) is recognised reliably; class 5 has the next-highest recall (0.58) but low precision (0.32, F1 0.41), and the remaining classes have F1 ≤0.33. Test accuracy is 39.5% and macro-F1 0.35, yet AUC is still 0.80, which means the ranking of class probabilities is

informative even when the arg-max decision is wrong.

C. Contribution of Each Component

Encoder. Here "encoder" means the trainable projection head (Linear + ReLU); the frozen pretrained feature extractor is covered in Section VI. The head re-maps the generic frozen features into a smaller task-specific space (256-d) shared by the attention branch and the pooled embedding, and it holds 60% of the parameters. *Aggregator.* It converts a variable-size bag into one fixed-size vector by weighting patches by learned importance; this is the only place where evidence from different patches is combined. *Classifier.* A single linear layer maps the slide embedding to class logits, so any non-linearity must come from the encoder and the gating.

To measure the aggregator, we retrained each model with all attention logits set to zero (uniform mean pooling), keeping everything else fixed (Table VI). AUC drops by 0.28, 0.19 and 0.14 on C16, C17 and BRACS and macro-F1 by 0.28, 0.34

and 0.14, so learned attention is associated with much better test performance in this setup. The gaps are large relative to the one-slide granularity of the test sets, but they come from a single seed and were not tested across seeds. On C17 the mean-pooling model reaches 60.5% accuracy, which equals the share of class 0 in the test set (121/200), and its macro-F1 of 0.19 matches predicting class 0 for every slide; its AUC of 0.644 shows that the probabilities still carry some ranking information, but its arg-max decisions are those of a majority-class predictor. This checkpoint was selected at epoch 1 on a 30-slide validation set. Because this comparator

is degenerate on C17 (majority-class decisions), the C17 macro-F1 gap of 0.34 should not be over-interpreted; AUC (0.839 vs 0.644) is the more informative comparison there.

TABLE VI
AGGREGATOR ABLATION (SINGLE RUN, SEED 42)

Metric	Attention	Mean pool
AUC	0.9342	0.6584
Macro-F1	0.8668	0.5883
Acc (%)	88.37	67.44
AUC	0.8391	0.6443
Macro-F1	0.5323	0.1885
Acc (%)	78.50	60.50
AUC	0.8042	0.6616
Macro-F1	0.3494	0.2103
Acc (%)	39.53	33.72

Two caveats apply to this ablation. First, the encoder and classifier were not ablated, so the experiment shows that learned attention helps but does not rank the three components by importance. Second, the mean-pooling model keeps the (now unused) attention weights and is trained with the same learning rate and number of epochs as the attention model; a different training budget might narrow the gap.

Attention statistics on the test slides give a complementary view. The average normalised attention entropy (1 = uniform) is 0.84 (C16), 0.72 (C17) and 0.74 (BRACS), so attention is only mildly concentrated by this measure on C16 and moderately concentrated on C17 and BRACS. The top 1% of patches receive on average 26%, 38% and 46% of the attention mass, i.e. roughly 26–46 times their uniform share.

D. Context: Published ABMIL Results

Table VII places our numbers next to published ABMIL results. These are *not* directly comparable with ours: the papers differ in feature extractors, splits, class definitions and metric conventions, and they typically report means over several runs, whereas we report one seed. Published ABMIL results on CAMELYON16 depend on the study protocol and patch encoder: the DSMIL study reports 84.5% accuracy and 0.865 AUC for ABMIL [23], and the MMIL-Transformer study reports 86.8% accuracy and 0.876 AUC for ABMIL with ResNet-50 features [49]. Our C16 AUC of 0.934 is higher than both, but the feature extractor, splits and number of runs

all differ, so we do not read this as evidence that our implementation is better. We do not know which factor explains the difference, and we did not test this. For our CAMELYON17 task (4-class slide-level metastasis type, center-based split) and for 7-class BRACS we found no published ABMIL results under the same protocol. We therefore do not claim that our baseline matches or exceeds published ABMIL.

TABLE VII
OUR SINGLE-RUN ABMIL VS. PUBLISHED ABMIL

This work	Published ABMIL (other pr
Acc 88.4%, AUC 0.934	DSMIL: Acc 84.5%, AUC 0.
	Transformer (ResNet-50): Ac
Acc 78.5%, AUC 0.839	No equivalent protocol found
Acc 39.5%, AUC 0.804	No 7-class ABMIL result fou

E. Caveats

All numbers come from one seed and small test sets with few slides per minority class, so differences of a few points (especially macro-F1 on C17 and BRACS) are not statistically reliable; multiple seeds would be needed for mean ± std. Only the aggregator was ablated; the encoder and classifier contributions in Section X-C are functional descriptions plus parameter counts, not measured effects. An identity encoder (attention directly on the 384-d features) and a classifier with a hidden layer would be the natural next ablations. The validation sets are also small (27, 30 and 62 slides, with as few as two slides for some classes), which makes checkpoint selection noisy. Attention weights show where the model looks, not causal evidence for the prediction. Any extension must reuse the same features, splits and seed to be comparable with these baselines.

XI. RESEARCH EXTENSIONS

A. Introduction

The baseline in Section X is our implementation of the gated attention-based multiple-instance learning (ABMIL) head [7] trained on frozen 384-d patch features. A single run (seed 42) reached AUC 0.934, 0.839 and 0.804 on CAMELYON16 (C16, two classes), CAMELYON17 (C17, four classes) and BRACS (seven classes). Three observations from that analysis motivate the extension studied here.

1) **Overfitting.** On BRACS the validation loss was lowest at epoch 4 and then rose from about 1.4 to 2.6, while training accuracy reached about 95% and validation accuracy stayed near 40%. C17 showed the same drift after epoch 9. 2) **Imbalance.** The loss weights every slide equally, whereas C17 has 177 class-0 but only 18 class-1 training slides. Test F1 for C17 classes 1 and 2 was 0.24

and 0.19, and macro-F1 (0.53) was far below accuracy (78.5%). 3) **Concentrated attention.** The top 1% of patches received 26%, 38% and 46% of the attention mass on C16, C17 and BRACS, yet the normalised entropy was still 0.72–0.84, so a lot of weight is spread thinly over many patches. Section XI-B states two proposals, Section XI-C explains which one was implemented, Sections XI-D and XI-E describe the experiments and results, and Section XI-F discusses them. The task rules require the same features, splits and seed as the baseline, and we follow them throughout.

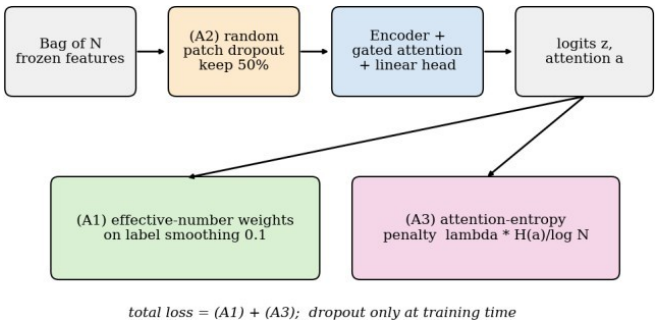


Fig. 5. Regularised ABMIL. The network is the baseline; only the training signal changes. Dropout is applied during training only.

B. Research Proposals

1) Idea A: Regularised ABMIL (implemented): Idea A keeps the baseline architecture unchanged (no new parameters) and modifies only how it is trained. Fig. 5 shows the three components, **A1–A3**, which can be enabled independently.

A1. Class-weighted label smoothing (effective-number weights). Motivation. Majority classes dominate the gradient of an unweighted loss (C17 imbalance 177:18, BRACS 119:24 among training slides). *Design.* The class weight of class c is the inverse effective number of samples of that class [50], $w_c \propto (1-\beta)/(1-\beta^{n(c)})$, where $n(c)$ is the number of training slides of class c, and the target distribution is smoothed with $\varepsilon = 0.1$ [51]. In the implementation $\beta = 0.999$, and the weights are rescaled to sum to the number of classes K, giving 0.21, 1.91, 0.94 and 0.94 for the four C17 classes. With batch size one (the setting used here), PyTorch's weighted cross-entropy with label smoothing gives the per-slide loss in Eq. (6). The weight of the true class cancels from the hard-label term, so the weights act only on the smoothing term. This differs from the class-balanced loss of [50], which multiplies the whole per-sample loss. A1 is therefore best described as class-weighted label smoothing rather than a class-balanced loss, and the imbalance motivation above applies only partly to it:

$$\mathcal{L}_{\text{cls}} = (1-\varepsilon)\,\mathrm{CE}(f(X), y) + \frac{\varepsilon}{K\,w_y}\sum_c w_c\,(-\log p_c) \quad (6)$$

Rationale. The effective-number weighting grows more slowly than plain inverse frequency, so classes with only a few dozen slides do not receive extreme weights. Because the weights act on the smoothing term only (Eq. (6)), they reshape the smoothing targets rather than the gradient on the correct class, so the implemented weighting is a weaker intervention than the class-balanced loss of [50]. Label smoothing targets the overconfidence visible in Section X, where training loss fell to 0.0001 (C16), 0.03 (C17) and 0.19 (BRACS) while validation loss rose, and it is known to affect calibration [52]. The smoothing term is also asymmetric across classes: with the C17 weights, a class-0 slide receives a smoothing coefficient $\varepsilon/(Kw_y) = 0.119$ that weights the other classes, especially class 1 ($w = 1.91$), whereas a class-1 slide receives only 0.013. The experiment therefore does not test whether a properly class-balanced loss (w_y times the per-slide loss) would help the minority classes; that is left as future work.

A2. Random patch dropout. Motivation. A slide has roughly 9–12k strongly redundant patches, but the model sees only 242–389 distinct training bags, each repeated 30 times in identical form, which invites memorisation; dropout-style noise is the classic remedy [53]. *Design.* At every training step a random fraction $\rho = 0.5$ of the patches of the slide is kept; the softmax of the gated attention is taken over the kept patches, and at validation and test time all patches are used. *Rationale.* Each epoch then presents a different sub-bag, which acts as instance-level augmentation. It resembles the random masking in ACMIL [25] but is not guided by attention and adds only one hyperparameter. The risk is that a small positive region can fall entirely outside the kept set, which would conflict with the standard MIL assumption for the smallest C17 classes.

A3. Attention-entropy penalty. Motivation. We want attention that is concentrated on a few patches that can be inspected by a pathologist. *Design.* With attention weights α_i over N kept patches, the normalised entropy $\tilde{H}(\alpha) = -\Sigma_i\ \alpha_i \log \alpha_i / \log N$, which lies in [0, 1] (1 = uniform), and the total loss adds it to the classification loss (Eq. (7)):

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda\,\tilde{H}(\alpha), \qquad \lambda = 0.1 \qquad (7)$$

Rationale. Entropy minimisation was introduced for classifier outputs on unlabelled data [54]; we apply the same idea, by analogy, to the attention distribution, where it favours peaked weights. The coefficient is deliberately small because the literature warns that too much concentration lets the model rely on a few easy patches [25].

Pre-stated hypotheses. These are the hypotheses in the project proposal (docs/proposals.md): (H1) higher macro-F1 on the imbalanced C17 and BRACS tasks; (H2) a smaller train/validation gap; (H3) lower attention entropy and higher top-1% mass; (H4) little change on C16, which is near saturation. Each is checked below for direction only: with one seed and differences of one to three slides, we do not claim that any hypothesis is confirmed or rejected.

2) Idea B: context-aware aggregator (proposal only): Motivation. ABMIL assigns each patch an attention score independently, so the pooled slide representation has no explicit notion of how patches relate to one another. *Design.* Before the gated attention we would insert one lightweight self-attention block (residual connection) on the encoded features, optionally with a TransMIL-style [26] positional module if patch coordinates are available in the feature files.

A zero-initialised residual gate would make the model start as the baseline, so the extra capacity is used only if it helps. We use a single block because the training sets are small (269–389 slides), each extra block adds parameters and another N × N attention cost, and one round of patch-to-patch interaction is enough to test whether context helps at all. The block works at the encoder's 256-d width so that no extra projection is needed and the pooled slide embedding keeps the baseline's dimension. *Cost and risk.* Full self-attention on $N \approx 10^4$ patches builds an $N \times N$ matrix of about 10^8 entries per head (roughly 0.4 GB in float32), which is too large for a T4 together with gradients. Two remedies are possible: attend over a random subset of $M \approx 1$–2k patches during training, or use Nyström landmarks [27] to reduce the cost to $O(N \cdot m)$. With 269–389 training slides the added parameters could also worsen the overfitting seen in Section X, so it would have to be compared with a baseline of equal tuning effort. *Planned evaluation.* Same features, splits and seed; report AUC, macro-F1 and parameter count against the baseline and against Idea A.

C. Choice of Idea

We implemented Idea A for three reasons. It addresses the overfitting and imbalance that Section X documented. It adds no parameters, so any change is due to the training signal. And it is cheap enough to run an ablation of each component on the available T4 GPU. Idea B needs a subsampling or Nyström design whose own hyperparameters would take a separate study.

D. Experimental Setup

Protocol. We reuse the starter data loader, the supplied splits (Table IV of Section X), the 384-d features, AdamW with learning rate 1e-4 and weight decay 1e-5, batch size one slide, 30 epochs, constant learning rate and seed 42. The checkpoint with the highest validation AUC + macro-F1 is restored and tested once. The baseline was re-run in the same session as the extension runs (not the session of the Section X baseline run); it reproduced the Section X test numbers exactly (C16 88.37% / 0.9342, C17 78.50% / 0.8391, BRACS 39.53% / 0.8042 accuracy / AUC), so the comparison is like for like. Validation loss is always logged as plain, unweighted cross-entropy so that curves of the baseline and the regularised model are comparable. The training loss in the logs includes the class weights, label smoothing and entropy term, so it is not comparable with the baseline's training loss.

Variants. *full* uses A1+A2+A3 ($\varepsilon = 0.1$, $\rho = 0.5$, $\lambda = 0.1$). The ablations cb_only (A1, class-weighted label smoothing; the name is kept from the code), drop_only (A2) and ent_only (A3) enable one component each. *full* was run on all three datasets; the ablations were run on C17 and BRACS only, to limit cost. The settings were chosen once and not tuned on the test sets.

Metrics. Accuracy, macro-F1, macro one-vs-rest AUC and cross-entropy on the test set, per-class F1, and the attention statistics of Section X (mean normalised entropy and top-1% mass on test slides, computed with all patches).

E. Results

1) Comparison with the baseline on all three datasets: Table VIII summarises the main result. On C16 and BRACS the regularised model classifies two and three more test slides correctly (1 slide is 0.78 and 1.16 points of accuracy there) and has a higher macro-F1. On C17 it classifies one slide fewer and macro-F1 drops by 0.048. AUC is lower on all three datasets, by 0.004–0.018, and the test cross-entropy is worse on C16 and C17 and better only on BRACS. There is therefore no consistent improvement. The direction expected by H4 (little change on C16) is observed: the two-slide accuracy change and the 0.009 AUC change are small, and both models reach validation AUC 1.0 by epoch 3. For H2 (smaller train/validation gap), the gap shrinks only on BRACS: the end-of-training accuracy gap falls from 53.2 to 50.4 points, while on C17 it grows from 22.6 to 28.9 points. Only accuracy gaps are compared, because the regularised training loss includes the penalty terms. The regularised training accuracy is also measured on random 50% sub-bags, whereas the baseline's is measured on full bags, so the two gaps are not strictly like for like.

TABLE VIII
TEST PERFORMANCE, BASELINE VS. REGULARISED ABMIL (SEED 42)

	Acc (%)	AUC	Macro-F1
	88.37	0.9342	0.8668
Regularised	89.92	0.9250	0.8860
	+1.55	−0.0092	+0.0192
	78.50	0.8391	0.5323
Regularised	78.00	0.8212	0.4841
	−0.50	−0.0179	−0.0482
	39.53	0.8042	0.3494
Regularised	43.02	0.8000	0.3705
	+3.49	−0.0042	+0.0211

Fig. 6 shows where the effect is largest. On BRACS the baseline validation loss climbs to 2.57 at epoch 30, while the regularised model ends at 1.68 and has a slightly lower minimum (1.34 at epoch 4 against 1.38). The loss curves show a smaller divergence after the minimum on BRACS; this observation goes beyond the stated hypotheses. On C17 the rise after the minimum is also smaller (0.37 for the baseline, from 0.43 at epoch 9 to 0.80 at epoch 30; 0.21 for the regularised model, from 0.62 at epoch 6 to 0.83), so the same observation holds on C17, although those curves are

noisy. The validation AUC of the regularised model nevertheless declines more after epoch 5 on BRACS and C17, so the lower loss is consistent with less confident predictions rather than better ranking. On C16 both models reach validation AUC 1.0 by epoch 3, and the regularised model keeps a small but non-zero validation loss instead of falling to zero.

2) Component ablation: On BRACS each single component gives the same 34 correct slides as the baseline (39.53%; Table IX), and only the combination reaches 37 (43.02%). We cannot tell from one seed whether this is a real interaction or a lucky checkpoint. On C17 the components move the metrics in different directions: cb_only (class-weighted smoothing) gives the best accuracy (80.0%) but the lowest macro-F1 of

all five variants (0.469, against 0.484 for the full model and 0.532 for the baseline), whereas patch dropout alone gives the highest macro-F1 (0.564, above the baseline) at the price of a higher loss. No component meaningfully improves AUC on either dataset; the largest gain is ent_only on BRACS, +0.0001 (0.80437 against 0.80424). Combining all three on C17 did not add up the individual benefits.

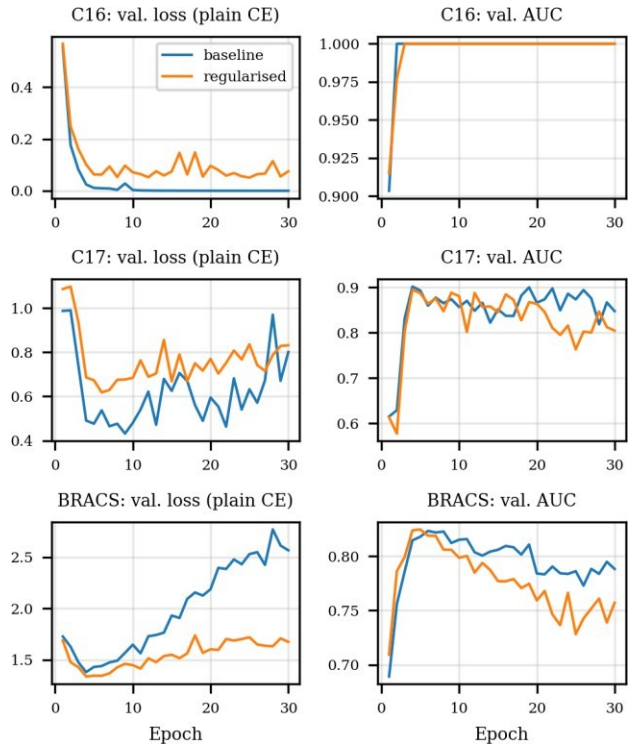


Fig. 6. Validation loss (plain CE, left) and validation AUC (right) per epoch for the baseline and the full regularised model. Rows are C16, C17 and BRACS, top to bottom.

TABLE IX
ABLATION OF THE THREE COMPONENTS (SEED 42)

Variant	Acc (%)	AUC	Macro-F1
Baseline	78.50	0.8391	0.5323
A1 (cb_only)	80.00	0.8232	0.4689
A2 (drop_only)	77.50	0.8250	0.5636
A3 (ent_only)	75.00	0.8154	0.5004
	78.00	0.8212	0.4841
	39.53	0.8042	0.3494
A1 (cb_only)	39.53	0.7807	0.3494
A2 (drop_only)	39.53	0.8026	0.3374
A3 (ent_only)	39.53	0.8044	0.3494
	43.02	0.8000	0.3705

3) Per-class behaviour and attention statistics: The BRACS gain comes mainly from classes 0, 2, 4 and 5, which improve (Table X), while class 3 falls from 0.31 to 0.12 and class 1 from 0.33 to 0.29. On C17 the weakest class, class 1 (the smallest class, 18 training slides), deteriorates further (0.24 to 0.10) although the class weights were intended to help it (they enter only the smoothing term, so they cannot raise the gradient on class-1 slides; see A1). We did not test why, but a plausible

reason is A2: with half of the patches removed, a very small positive region is often absent from the kept set, so the slide appears negative to the model during training. For H1 (higher macro-F1), the direction matches on BRACS and is opposite on C17; with one seed neither result is established.

TABLE X
PER-CLASS TEST F1 (BASELINE → REGULARISED)

	F1 for classes 0, 1, 2, . . .
	0.914→0.925, 0.819→0.847
	0.922→0.902, 0.240→0.095, 0.188→0.154, 0.780→0.785
	0.222→0.364, 0.333→0.286, 0.125→0.200, 0.308→0.118,
	0.412→0.500, 0.811→0.811

TABLE XI
ATTENTION STATISTICS ON TEST SLIDES

Method	Norm. entropy	Top-1% mas
Baseline	0.838	0.263
Regularised	0.813	0.289
Baseline	0.720	0.385
Regularised	0.754	0.399
Baseline	0.736	0.455
Regularised	0.714	0.473

In Table XI, attention became somewhat more concentrated on C16 and BRACS (lower entropy, higher top-1% mass) but the changes are small (at most 0.03 in mass), and on C17 the entropy actually rose while the top-1% mass increased slightly. The direction expected by H3 is therefore observed on C16 and BRACS but not fully on C17, where entropy rose, and the changes are small. Possible reasons are the small λ and the mismatch between the sub-bags used in training and the full bags used at test time, but we did not isolate either. The extra training time is about 10–25 s per run (219→242 s on C16, 257→282 s on C17, 365→375 s on BRACS), so the method is practically free.

F. Discussion

What the evidence supports. In this seed, the full variant shows a smaller rise of plain validation cross-entropy after its minimum on BRACS (2.57 vs 1.68 at epoch 30), and its best epoch moves earlier (BRACS 6→4, C17 22→9). This is not general across variants: on C17 the best epoch is 7 for cb_only, 26 for drop_only and 18 for ent_only, and on BRACS every single-component variant selects epoch 6, so the earlier best epoch is a property of the combination in one seed. The lower loss did not translate into reliably better discrimination.

Why the result is inconclusive. All numbers come from one seed. One slide changes C17 accuracy by 0.5 points and BRACS accuracy by 1.2 points, so the gains of two and three slides may be within run-to-run variation, which we did not measure, and the validation sets contain as few as two slides for some classes, which makes the selected epoch itself noisy. We did not run significance tests or compute intervals for AUC and macro-F1 (per-slide probabilities were not saved). Wilson 95% intervals on accuracy overlap almost completely (regularised vs baseline: C16 83.5–94.0% vs 81.7–92.8%, C17 71.8–83.2% vs 72.3–83.6%, BRACS 33.1–53.6% vs 29.9– 50.1%), so we make no claim that the extension is better or worse than the baseline.

Likely mechanisms (not tested). Lower validation loss with lower validation AUC is consistent with label smoothing and the smoothing-term weights mainly changing confidence rather than ranking. The worsening of C17 class 1 is consistent with patch dropout removing tiny positive regions. These are hypotheses; isolating them would need runs that vary ρ and ε separately over several seeds.

Fairness. The baseline and the extension share the same code path, features, splits, optimiser, epoch budget and selection rule. Neither was tuned. A tuned baseline (for example with ordinary dropout or weight decay) was not compared, so the effect of the new components relative to simple regularisation remains open.

G. Limitations and Future Work

- Single seed and no variance estimate; the first priority is training seeds 42–44 with mean ± std (the split seed stays 42 so the splits are unchanged) and bootstrap intervals on the test sets.
- Ablations were not run on C16, and the values $\rho = 0.5$, $\varepsilon = 0.1$ and $\lambda = 0.1$ were not swept. A larger keep fraction ρ (less dropout), possibly class-dependent for classes with small lesions, or an attention-guided mask as in ACMIL, may avoid the C17 class-1 failure.
- Attention sparsity was measured only as entropy and top-1% mass; a deletion test with the C16 tumour annotations would be needed to say whether the sparser attention is more faithful [46].
- Idea B remains unimplemented; a subsampled self-attention block initialised as an identity is the natural next experiment.

H. Conclusion

We proposed two extensions of ABMIL and implemented the training-side one, Regularised ABMIL, on all three datasets. Under an identical protocol, in this single seed, it shows a smaller rise of BRACS validation loss and gives small accuracy and macro-F1 gains on C16 and BRACS, but it lowers AUC everywhere and hurts the smallest C17 classes. With one seed and small test sets, the honest summary is a mixed, inconclusive result that motivates a multi-seed study and a gentler form of patch dropout (a larger keep fraction).

APPENDIX A
MACHINE LEARNING PAPER REPRODUCIBILITY CHECKLIST (V2.0)

This is the Machine Learning Paper Reproducibility Checklist, v2.0 (https://www.cs.mcgill.ca/~jpineau/ReproducibilityChecklist.pdf). It covers this report (Table XII).

Items 11–15 concern the code artefact, which is a separate artefact with its own checklist, docs/reproducibility_checklist.md, named in the repository README.

TABLE XII
MACHINE LEARNING PAPER REPRODUCIBILITY CHECKLIST (V2.0)

	Item	Resp.
For all models and algorithms presented		
1.	A clear description of the mathematical setting, algorithm, and/or model.	Yes
2.	A clear explanation of any assumptions.	Yes
3.	An analysis of the complexity (time, space, sample size) of any algorithm.	Yes
For any theoretical claim		
4.	A clear statement of the claim.	N/A
5.	A complete proof of the claim.	N/A
For all datasets used		
6.	The relevant statistics, such as number of examples.	Yes
7.	The details of train / validation / test splits.	Yes
8.	An explanation of any data that were excluded, and all pre-processing steps.	Yes

9.	A link to a downloadable version of the dataset or simulation environment.	Yes
10.	For new data collected, a complete description of the data collection process.	N/A
For all shared code related to this work (answered from the repository checklist, docs/reproducibility_checklist.md)		
11.	Specification of dependencies.	Yes
12.	Training code.	Yes
13.	Evaluation code.	Yes
14.	(Pre-)trained model(s).	Yes
15.	README file includes a table of results accompanied by a precise command to run to produce those results.	No
For all reported experimental results		
16.	The range of hyper-parameters considered, method to select the best hyper-parameter configuration, and specification of all hyper-parameters used.	Yes
17.	The exact number of training and evaluation runs.	Yes
18.	A clear definition of the specific measure or statistics used to report results.	Yes
19.	A description of results with central tendency (e.g. mean) and variation (e.g. error bars).	No
20.	The average runtime for each result, or estimated energy cost.	Yes
21.	A description of the computing infrastructure used.	Yes

REFERENCES

[1] A. Madabhushi and G. Lee, “Image analysis and machine learning in digital pathology: Challenges and opportunities,” Med. Image Anal., vol. 33, pp. 170–175, 2016.

[2] T. G. Dietterich, R. H. Lathrop, and T. Lozano-Pérez, “Solving the multiple instance problem with axis-parallel rectangles,” Artif. Intell., vol. 89, no. 1–2, pp. 31–71, 1997.

[3] N. Weidmann, E. Frank, and B. Pfahringer, “A two-level learning method for generalized multi-instance problems,” in Proc. Eur. Conf. Mach. Learn. (ECML), ser. LNCS, vol. 2837, 2003, pp. 468–479.

[4] J. Foulds and E. Frank, “A review of multi-instance learning assumptions,” Knowl. Eng. Rev., vol. 25, no. 1, pp. 1–25, 2010.

[5] N. Brancati et al., “BRACS: A dataset for BReAst carcinoma subtyping in H&E histology images,” Database, vol. 2022, Art. no. baac093, 2022.

[6] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. J. Smola, “Deep sets,” in Proc. Adv. Neural Inf. Process. Syst. (NIPS), vol. 30, 2017, pp. 3391–3401.

[7] M. Ilse, J. Tomczak, and M. Welling, “Attention-based deep multiple instance learning,” in Proc. Int. Conf. Mach. Learn. (ICML), 2018, pp. 2127–2136.

[8] J. Amores, “Multiple instance classification: Review, taxonomy and comparative study,” Artif. Intell., vol. 201, pp. 81–105, 2013.

[9] M.-A. Carbonneau, V. Cheplygina, E. Granger, and G. Gagnon, “Multiple instance learning: A survey of problem characteristics and applications,” Pattern Recognit., vol. 77, pp. 329–353, 2018.

[10] V. Cheplygina, M. de Bruijne, and J. P. W. Pluim, “Not-so-supervised: A survey of semi-supervised, multi-instance, and transfer learning in medical image analysis,” Med. Image Anal., vol. 54, pp. 280–296, 2019.

[11] M. Gadermayr and M. Tschuchnig, “Multiple instance learning for digital pathology: A review of the state-of-the-art, limitations & future potential,” Comput. Med. Imaging Graph., vol. 112, Art. no. 102337, 2024.

[12] O. Maron and T. Lozano-Pérez, “A framework for multiple-instance learning,” in Proc. Adv. Neural Inf. Process. Syst. (NIPS), vol. 10, 1998, pp. 570–576.

[13] J. Wang and J.-D. Zucker, “Solving the multiple-instance problem: A lazy learning approach,” in Proc. Int. Conf. Mach. Learn. (ICML), 2000, pp. 1119–1126.

[14] T. Gärtner, P. A. Flach, A. Kowalczyk, and A. J. Smola, “Multi-instance kernels,” in Proc. Int. Conf. Mach. Learn. (ICML), 2002, pp. 179–186.

[15] S. Andrews, I. Tsochantaridis, and T. Hofmann, “Support vector machines for multiple-instance learning,” in Proc. Adv. Neural Inf. Process. Syst. (NIPS), vol. 15, 2003, pp. 577–584.

[16] X. Wang, Y. Yan, P. Tang, X. Bai, and W. Liu, “Revisiting multiple instance neural networks,” Pattern Recognit., vol. 74, pp. 15–24, 2018.

[17] P. O. Pinheiro and R. Collobert, “From image-level to pixel-level labeling with convolutional networks,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2015, pp. 1713–1721.

[18] L. Hou, D. Samaras, T. M. Kurc, Y. Gao, J. E. Davis, and J. H. Saltz, “Patch-based convolutional neural network for whole slide tissue image classification,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 2424–2433.

[19] G. Campanella et al., “Clinical-grade computational pathology using weakly supervised deep learning on whole slide images,” Nat. Med., vol. 25, no. 8, pp. 1301–1309, 2019.

[20] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language modeling with gated convolutional networks,” in Proc. Int. Conf. Mach. Learn. (ICML), 2017, pp. 933–941.

[21] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2015.

[22] M. Y. Lu, D. F. K. Williamson, T. Y. Chen, R. J. Chen, M. Barbieri, and F. Mahmood, “Data-efficient and weakly supervised computational pathology on whole-slide images,” Nat. Biomed. Eng., vol. 5, no. 6, pp. 555–570, 2021.

[23] B. Li, Y. Li, and K. W. Eliceiri, “Dual-stream multiple instance learning network for whole slide image classification with self-supervised contrastive learning,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2021, pp. 14318–14328.

[24] H. Zhang et al., “DTFD-MIL: Double-tier feature distillation multiple instance learning for histopathology whole slide image classification,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 18802–18812.

[25] Y. Zhang, H. Li, Y. Sun, S. Zheng, C. Zhu, and L. Yang, “Attention-challenging multiple instance learning for whole slide image classification,” in Proc. Eur. Conf. Comput. Vis. (ECCV), 2024.

[26] Z. Shao et al., “TransMIL: Transformer based correlated multiple instance learning for whole slide image classification,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 34, 2021, pp. 2136–2147.

[27] Y. Xiong et al., “Nyströmformer: A Nyström-based algorithm for approximating self-attention,” in Proc. AAAI Conf. Artif. Intell., vol. 35, 2021, pp. 14138–14148.

[28] X. Chu et al., “Conditional positional encodings for vision transformers,” arXiv:2102.10882, 2021.

[29] W. Tang, F. Zhou, S. Huang, X. Zhu, Y. Zhang, and B. Liu, “Feature re-embedding: Towards foundation model-level performance in computational pathology,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2024.

[30] S. Yang, Y. Wang, and H. Chen, “MambaMIL: Enhancing long sequence modeling with sequence reordering in computational pathology,” in Proc. Med. Image Comput. Comput.-Assist. Intervent. (MICCAI), 2024.

[31] R. J. Chen et al., “Scaling vision transformers to gigapixel images via hierarchical self-supervised learning,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 16144–16155.

[32] Y. Zhao et al., “Predicting lymph node metastasis using histopathological images based on multiple instance learning with deep graph convolution,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 4837–4846.

[33] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770–778.

[34] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in Proc. Int. Conf. Mach. Learn. (ICML), 2020, pp. 1597–1607.

[35] O. Ciga, T. Xu, and A. L. Martel, “Self supervised contrastive learning for digital histopathology,” Mach. Learn. Appl., vol. 7, Art. no. 100198, 2022.

[36] M. Kang, H. Song, S. Park, D. Yoo, and S. Pereira, “Benchmarking self-supervised learning on diverse pathology datasets,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023, pp. 3344–3354.

[37] M. Caron et al., “Emerging properties in self-supervised vision transformers,” in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2021, pp. 9650–9660.

[38] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2021.

[39] R. J. Chen et al., “Towards a general-purpose foundation model for computational pathology,” Nat. Med., vol. 30, pp. 850–862, 2024.

[40] M. Y. Lu et al., “A visual-language foundation model for computational pathology,” Nat. Med., vol. 30, pp. 863–874, 2024.

[41] E. Vorontsov et al., “A foundation model for clinical-grade computational pathology and rare cancers detection,” Nat. Med., vol. 30, pp. 2924–2935, 2024.

[42] H. Xu et al., “A whole-slide foundation model for digital pathology from real-world data,” Nature, vol. 630, pp. 181–188, 2024.

[43] B. Ehteshami Bejnordi et al., “Diagnostic assessment of deep learning algorithms for detection of lymph node metastases in women with breast cancer,” JAMA, vol. 318, no. 22, pp. 2199–2210, 2017.

[44] P. Bandi et al., “From detection of individual metastases to classification of lymph node status at the patient level: The CAMELYON17 challenge,” IEEE Trans. Med. Imag., vol. 38, no. 2, pp. 550–560, 2019.

[45] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2019.

[46] S. Jain and B. C. Wallace, “Attention is not explanation,” in Proc. NAACL-HLT, 2019, pp. 3543–3556.

[47] S. Wiegreffe and Y. Pinter, “Attention is not not explanation,” in Proc. EMNLP-IJCNLP, 2019, pp. 11–20.

[48] A. F. T. Martins and R. F. Astudillo, “From softmax to sparsemax: A sparse model of attention and multi-label classification,” in Proc. Int. Conf. Mach. Learn. (ICML), 2016, pp. 1614–1623.

[49] R. Zhang et al., “Multi-level multiple instance learning with transformer for whole slide image classification,” arXiv:2306.05029, 2023.

[50] Y. Cui et al., “Class-balanced loss based on effective number of samples,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2019, pp. 9268–9277.

[51] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the Inception architecture for computer vision,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 2818–2826.

[52] R. Müller, S. Kornblith, and G. Hinton, “When does label smoothing help?” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 32, 2019.

[53] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” J. Mach. Learn. Res., vol. 15, pp. 1929–1958, 2014.

[54] Y. Grandvalet and Y. Bengio, “Semi-supervised learning by entropy minimization,” in Proc. Adv. Neural Inf. Process. Syst. (NIPS), vol. 17, 2004, pp. 529–536.